Java Testing & Quality Guidelines

DevPilot AI Knowledge Base • Java 21 • Spring Boot • JUnit 5 • Mockito

Purpose
A practical guide for designing trustworthy tests, finding defects early, and maintaining code quality in Java and Spring Boot services. The literal “Heading:” and “Section:” prefixes are intentional retrieval anchors for document chunking and RAG.

Heading: 1. Testing Principles

Section: Tests should protect behavior

- Test externally observable behavior and business rules, not private implementation details.
- A useful test has a clear setup, action, and assertion. Keep each test focused on one behavior.
- Prefer deterministic tests: no uncontrolled network calls, current-time assumptions, random data, or shared mutable state.
- A test suite should be readable enough that a failure explains what broke.

Section: The test pyramid is a guide, not a quota

- Unit tests: fast, isolated checks of a class or small unit; use these for business logic and edge cases.
- Slice tests: load a focused Spring layer, such as MVC or JPA, without booting every component.
- Integration tests: verify several real components work together, such as application configuration, persistence, and HTTP wiring.
- End-to-end tests: validate critical user journeys across deployed boundaries. Keep these limited because they are slower and more failure-prone.

Aim for many fast tests and a smaller number of broad integration tests. Choose the test level based on the risk being verified.

Heading: 2. Java Test Stack and Project Layout

Section: Common tools

- **JUnit Jupiter (JUnit 5)** for test lifecycle, assertions, parameterized tests, and extensions.
- **Mockito** for replacing collaborators in isolated unit tests and verifying meaningful interactions.
- **Spring Boot Test** for application-context tests and Spring test utilities.
- **MockMvc** for exercising Spring MVC request handling without starting a real HTTP server.
- **AssertJ** for fluent, readable assertions, when included in the project.
- **Testcontainers** when integration tests need a real database or service dependency in a disposable container.

Section: Keep test sources parallel to production sources

```
src/
  main/java/com/example/orders/
    controller/OrderController.java
    service/OrderService.java
    repository/OrderRepository.java
    domain/Order.java
  test/java/com/example/orders/
    controller/OrderControllerTest.java
    service/OrderServiceTest.java
    repository/OrderRepositoryTest.java
```

Use the same package structure where practical. Keep test-only builders, fixtures, and helpers under src/test/java; do not ship them in the production artifact.

Heading: 3. Naming, Structure, and Assertions

Section: Use behavior-oriented test names

```
@Test
void rejectsOrderWhenQuantityIsZero() { ... }

@Test
void returnsNotFoundWhenOrderDoesNotExist() { ... }
```

Section: Arrange, Act, Assert

```
@Test
void appliesDiscountForEligibleCustomer() {
    // Arrange
    Customer customer = new Customer(true);
    Price price = new Price(new BigDecimal("100.00"));

    // Act
    Price result = pricingService.finalPrice(customer, price);

    // Assert
    assertThat(result.amount()).isEqualByComparingTo("90.00");
}
```

- Use descriptive assertion messages only when the assertion itself does not make the failure clear.
- Assert exact outcomes and important state changes. Avoid asserting incidental call order or private data structures.
- Avoid multiple unrelated scenarios in one test. Parameterized tests are useful when the same rule has several input/output cases.
- Prefer BigDecimal comparisons for money; define scale and rounding rules explicitly in production code and tests.

Heading: 4. Unit Testing with Mockito

Section: Mock collaborators, not the class under test

```
@ExtendWith(MockitoExtension.class)
class OrderServiceTest {
    @Mock OrderRepository orderRepository;
    @Mock PaymentGateway paymentGateway;
    @InjectMocks OrderService orderService;

    @Test
    void createsOrderAfterPaymentSucceeds() {
        CreateOrder command = new CreateOrder("customer-7", 2);
        when(paymentGateway.charge(any())).thenReturn(PaymentResult.approved());

        Order result = orderService.create(command);

        assertThat(result).isNotNull();
        verify(paymentGateway).charge(any());
        verify(orderRepository).save(any(Order.class));
    }
}
```

Section: Mocking rules of thumb

- Mock external collaborators at a unit-test boundary; use real value objects and simple domain objects.
- Stub only the behavior needed by the scenario. Over-stubbing makes tests brittle.
- Verify interactions when they are part of the contract, such as charging a payment exactly once; do not verify every getter or internal method.
- Test both success and failure paths: declined payment, repository failure where relevant, and validation rejection.

- Avoid mocking the repository in a test whose goal is to prove a database query or mapping works; use a persistence integration test instead.

The code above is illustrative. Adapt constructor signatures, domain types, and test dependencies to the actual project.

Heading: 5. Testing Spring MVC and REST APIs

Section: MVC slice tests with MockMvc

```
@WebMvcTest(OrderController.class)
class OrderControllerTest {
    @Autowired MockMvc mockMvc;
    @MockBean OrderService orderService;

    @Test
    void returnsBadRequestForInvalidRequest() throws Exception {
        mockMvc.perform(post("/api/orders")
                .contentType(MediaType.APPLICATION_JSON)
                .content("{\"customerId\":\"\",\"quantity\":0}"))
            .andExpect(status().isBadRequest());
    }
}
```

- Check status code, response body shape, relevant headers, validation errors, and JSON serialization.
- Cover valid requests, malformed JSON, missing required fields, invalid values, unknown IDs, and expected service errors.
- Keep controller tests focused on HTTP mapping and validation. Test business decisions primarily in service tests.
- For secured endpoints, include authorization tests for unauthenticated, unauthorized, and permitted users.
- Use the Spring Security test support available in the project's dependency set when testing authentication and roles.

Spring Boot versions differ in test annotations and mock-bean APIs. Follow the version actually used by the project; do not copy annotations blindly.

Heading: 6. Persistence and Database Tests

Section: Test persistence behavior against the intended database

- Repository tests should verify query predicates, sorting, pagination, entity relationships, and constraints.
- An embedded database can be useful for fast tests, but it may not match production SQL behavior. Use the same database engine in integration tests when compatibility matters.
- Test schema migrations from a clean database and, where relevant, upgrade paths from supported prior versions.
- Verify transaction boundaries and rollback behavior when these are part of the requirement.
- Use isolated test data. Avoid depending on rows left by another test.

Section: Data setup and cleanup

- Prefer builders or small fixture methods that create valid domain objects.
- Use unique keys or per-test database isolation to avoid collisions during parallel runs.
- Do not disable constraints simply to make test setup easier; create data that respects the production model.
- Keep tests independent of execution order. A test should pass alone and in the full suite.

Heading: 7. What to Test: Risk-Based Coverage

Section: Business and domain logic

- Happy path and meaningful alternate paths.
- Boundary values: zero, one, maximum supported value, empty input, and just-outside-limit values.
- Null or absent values where the API contract permits them; reject them clearly where it does not.

- Rounding, date/time zones, duplicate requests, idempotency, and concurrent updates when relevant to the domain.

Section: API and application behavior

- Input validation and stable error response format.
- Authentication and role/permission checks.
- Pagination defaults and limits; filtering and sorting rules.
- Dependency failures, timeout behavior, retry policies, and fallback behavior when configured.

Section: Security-sensitive behavior

- Authorization must be checked on server-side boundaries, not only in the UI.
- Ensure invalid identifiers cannot access another user's resources.
- Test that secrets and sensitive fields are not returned in responses or logs.
- Test input handling for injection-prone fields using parameterized database access and validation.

Coverage percentage is a signal, not proof of quality. A high line-coverage score can still miss incorrect assertions, untested branches, or broken requirements.

Heading: 8. Parameterized Tests and Edge Cases

Section: Use parameterized tests for data-driven rules

```
@ParameterizedTest
@CsvSource({
    "1, true",
    "10, true",
    "0, false",
    "-1, false"
})
void acceptsOnlyPositiveQuantities(int quantity, boolean expected) {
    assertThat(quantity > 0).isEqualTo(expected);
}
```

- Include values at the boundary and one value on either side of the boundary.
- Keep test cases named or documented so failures identify the input scenario.
- For complex input, use MethodSource with readable domain objects instead of a long, opaque CSV string.
- Do not add edge cases without a reason; prioritize cases implied by requirements, invariants, and past defects.

Heading: 9. Integration Tests and External Dependencies

Section: Choose the smallest realistic environment

- Use a full Spring application context only when the wiring or configuration itself is under test.
- Use Testcontainers for database behavior that depends on the real database engine, SQL dialect, or extension.
- Stub external HTTP services at a clear boundary unless the test specifically validates the integration contract.
- Test serialization, timeouts, authentication configuration, and error translation for important service integrations.
- Never require production credentials or mutate production data from a test suite.

Section: Keep integration tests repeatable

- Pin container or dependency versions where practical.
- Use bounded timeouts and reliable readiness checks.
- Make cleanup explicit and ensure tests can run repeatedly.
- Separate tests requiring Docker or external infrastructure from the fast unit-test feedback loop if needed.

Heading: 10. Test Doubles: Fake, Stub, Mock

Section: Use the simplest test double that fits

- **Stub:** returns predetermined results, such as an approved payment.
- **Mock:** also records interactions so a test can verify a meaningful call.
- **Fake:** a lightweight working implementation, such as an in-memory repository, useful when it faithfully models the needed behavior.
- Do not treat an in-memory fake as proof that SQL queries, transactions, indexes, or database constraints work.
- Prefer real objects for simple values and deterministic domain logic. Too many mocks can make tests mirror the implementation rather than validate the requirement.

Heading: 11. Flaky Tests and Common Anti-Patterns

Section: Avoid

- Sleeping for an arbitrary duration to wait for asynchronous work; use condition-based waits and bounded timeouts.
- Tests that depend on test order, machine timezone, locale, current date, network availability, or shared files.
- Catching and ignoring exceptions in tests.
- Assertions that only check a result is non-null when a precise expected outcome is available.
- Excessive mocking, broad integration contexts for simple logic, and tests that duplicate implementation details.
- Random test data without a recorded seed or a way to reproduce the failure.

Section: When a test fails

- First determine whether the failure is a product defect, an incorrect test assumption, or an environment problem.
- Reproduce the failure in isolation, then in the full suite.
- Fix the underlying race, shared state, or timing assumption instead of adding retries that conceal the defect.
- Record a regression test for a confirmed bug, and ensure the test would fail against the faulty behavior.

Heading: 12. Coverage, Static Analysis, and Quality Gates

Section: Use several quality signals

- Line and branch coverage help reveal unexecuted code, but coverage alone does not measure assertion quality.
- Use the project's configured static analysis and formatting tools consistently.
- Treat compiler warnings, dependency vulnerabilities, and failing tests as review items rather than ignoring them by default.
- For critical business rules, consider mutation testing if the project supports it: it can reveal tests that execute code but fail to detect small behavioral changes.
- Set quality thresholds based on project risk and baseline. Avoid arbitrary thresholds that encourage meaningless tests.

Section: A useful pull-request gate

- Compile and run fast unit tests.
- Run targeted slice tests and integration tests appropriate to the change.
- Run formatting, static analysis, and dependency/security checks configured by the repository.
- Publish clear test results and fail the build when required checks fail.
- Keep test commands documented so developers can reproduce CI failures locally.

Heading: 13. Suggested Maven Workflow

Section: Typical commands

```
# Compile production and test sources, run tests
./mvnw test

# Run a single test class
./mvnw -Dtest=OrderServiceTest test

# Package the application (the project's lifecycle may run tests)
./mvnw package

# Skip test execution only for a deliberate, documented reason
./mvnw -DskipTests package
```

Use the Maven Wrapper when the repository provides it. Confirm plugin configuration and profiles before assuming these commands represent every CI stage. Skipping tests should not become the normal development workflow.

Heading: 14. Definition of Done for a Code Change

- Requirements and expected behavior are clear.
- Happy path and relevant negative paths are covered.
- Boundary conditions and validation rules are tested.
- Tests assert meaningful outcomes, not only implementation details.
- Database and security behavior are tested at an appropriate level.
- The full relevant test suite passes locally or in CI.
- No secrets, production data, or uncontrolled external calls are required.
- Failures are deterministic and diagnosable.
- New tests follow existing naming, package, and build conventions.
- The change has been reviewed for maintainability and risk.

Heading: 15. DevPilot RAG Retrieval Notes

Section: Recommended retrieval behavior

- Treat each Heading: block as a broad topic and each Section: block as a focused retrieval unit.
- For prompts about a specific layer, retrieve the narrowest relevant sections first: unit tests, MVC, persistence, security, or CI.
- Combine retrieved guidance with the repository's actual Java, Spring Boot, JUnit, Mockito, and build-tool versions.
- If the repository's conventions conflict with a generic example here, verify the codebase and project requirements before recommending a change.
- Use this guide as engineering guidance, not as evidence that any particular project already implements these practices.